



**Visuel à remplacer**

**UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD**

# Analyse de durabilité des systèmes de propulsion par microcontrôleurs externe

Rapport de stage STXX – P20XX

**NOM Prénom**

Nom de la spécialité d'ingénieur  
Nom de la filière

**Entreprise DEMO-Contrôleurs**

8 rue de la Fierté  
75 013 Paris  
[www.democontroleurs.com](http://www.democontroleurs.com)

Tuteur en entreprise  
**NOM Prénom**

Suiveur UTBM  
**NOM Prénom**

Tuteur en entreprise  
**JACQUES Pierre-Paul**

# Introduction

Dans le cadre de la dernière année du cursus d'ingénieur en informatique à l'Université de Technologie de Belfort-Montbéliard (UTBM), la réalisation d'un stage de fin d'études en immersion professionnelle constitue une étape charnière. Ce projet de fin d'études, d'une durée de six mois, offre l'opportunité d'appliquer et de consolider des compétences techniques et méthodologiques au sein d'un environnement industriel exigeant. C'est dans cette dynamique de transition vers la vie active que s'inscrit ce stage, effectué du 9 février au 24 juillet 2026 au sein de la société Magellium.

Fondée sur une expertise pointue dans le domaine de l'ingénierie logicielle, du traitement d'images et de la géomatique, Magellium s'impose comme un acteur de référence dans le développement de solutions technologiques innovantes. Intégré au cœur des équipes de l'agence de Ramonville-Saint-Agne, à proximité de Toulouse, ce stage s'est déroulé au sein d'un pôle technique où les problématiques de scalabilité, d'automatisation et d'architecture logicielle sont quotidiennes.

L'objectif principal de cette mission consistait à répondre à des défis technologiques d'actualité, notamment l'intégration et l'optimisation d'architectures modernes. Les travaux menés se sont articulés autour de la mise en œuvre de solutions full-stack et DevOps, avec une ouverture prononcée vers les technologies émergentes d'orchestration d'intelligence artificielle et d'agents autonomes. Ce projet a nécessité une immersion complète dans des environnements de développement agiles, exigeant rigueur, autonomie et force de proposition.

Ce rapport s'attache à présenter de manière détaillée l'ensemble des travaux réalisés au cours de ces six mois. Dans un premier temps, nous introduirons le contexte général de l'entreprise Magellium ainsi que son positionnement stratégique. Nous détaillerons ensuite l'analyse du besoin et les choix d'architecture technique qui ont guidé nos développements. Enfin, nous présenterons les phases d'implémentation, les résultats obtenus, ainsi qu'un bilan technique et personnel de cette expérience enrichissante.

## Remerciement

Je tiens à exprimer ma profonde gratitude à toute l'équipe de Magellium pour leur accueil bienveillant. Dès le premier jour, j'ai été intégré au sein d'une équipe dynamique et compétente, qui m'a offert un environnement de travail stimulant et propice à l'apprentissage. Je remercie particulièrement mon tuteur de stage, Mathieu Niord, pour son encadrement attentif, ses conseils avisés et sa disponibilité tout au long de ce stage. Sa guidance m'a permis de progresser rapidement et de surmonter les défis techniques rencontrés. Je suis également reconnaissant envers mes collègues pour leur soutien, leur collaboration et les échanges enrichissants qui ont jalonné cette expérience.

Cette expérience professionnelle a été très enrichissante, tant sur le plan personnel que technique. Elle m'a permis de développer de nouvelles compétences, d'approfondir mes connaissances et de me confronter à des problématiques réelles du monde industriel. Je suis convaincu que les enseignements tirés de ce stage me seront précieux pour la suite de ma carrière d'ingénieur en informatique.

Grâce à cette expérience, j'ai renforcé ma capacité d'exprimer mes idées, ma prise de décision et mon autonomie. Des atouts essentiels pour ma future carrière professionnelle. Je suis également reconnaissant envers l'Université de Technologie de Belfort-Montbéliard (UTBM) pour m'avoir offert cette opportunité de stage, qui a été une étape cruciale dans mon parcours académique et professionnel.

# Glossaire

Ce glossaire a pour but de définir les termes techniques utilisés dans ce rapport. Il est destiné à aider les lecteurs à comprendre les concepts et les technologies abordés.

- **Full-stack** : Se réfère à la capacité de travailler sur l'ensemble des couches d'une application, du front-end (interface utilisateur) au back-end (logique métier et base de données).
- **DevOps** : Un ensemble de pratiques qui combine le développement logiciel (Dev) et les opérations informatiques (Ops) pour améliorer la collaboration, l'automatisation et la livraison continue des applications.
- **Orchestration d'intelligence artificielle** : La coordination et la gestion de différentes composantes d'une solution d'intelligence artificielle, telles que les modèles, les données et les processus, pour atteindre des objectifs spécifiques.
- **Agents autonomes** : Des systèmes informatiques capables de prendre des décisions et d'agir de manière indépendante pour accomplir des tâches spécifiques, souvent en utilisant des techniques d'intelligence artificielle.
- **Scalabilité** : La capacité d'un système à gérer une augmentation de la charge de travail ou à être facilement étendu pour accueillir plus d'utilisateurs ou de données.
- **Automatisation** : L'utilisation de technologies pour effectuer des tâches sans intervention humaine, souvent pour améliorer l'efficacité et réduire les erreurs.
- **Architecture logicielle** : La structure organisationnelle d'un système logiciel, définissant les composants, leurs interactions et les principes de conception qui guident le développement.
- **Agile** : Une méthodologie de développement logiciel qui privilégie la collaboration, la flexibilité et l'adaptation aux changements tout au long du projet.
- **Rigueur** : La qualité d'être précis, méthodique et attentif aux détails dans l'exécution d'une tâche ou d'un projet.
- **Autonomie** : La capacité de travailler de manière indépendante, en prenant des décisions et en gérant son propre temps et ses ressources pour atteindre des objectifs définis.
- **Force de proposition** : La capacité à suggérer des idées, des solutions ou des améliorations de manière proactive pour contribuer au succès d'un projet ou d'une organisation.
- **RAG (Retrieval-Augmented Generation)** : Une technique d'intelligence artificielle qui combine la génération de texte avec la récupération d'informations à partir de sources externes pour produire des réponses plus précises et informées.

- **LLM (Large Language Model)** : Un modèle de langage de grande taille, souvent basé sur des architectures de réseaux de neurones, capable de comprendre et de générer du texte de manière cohérente et contextuelle.
- **Pipelines de données** : Un ensemble de processus automatisés qui permettent de collecter, transformer et stocker des données pour les rendre utilisables dans des applications ou des analyses.
- **MCP (Model-Context-Protocol)** : Un protocole de communication entre un modèle d'intelligence artificielle, et des outils mis à sa disposition, permettant de structurer les interactions et les échanges d'informations de manière efficace.

# Table des matières

|                                                                                            |    |
|--------------------------------------------------------------------------------------------|----|
| Introduction .....                                                                         | 2  |
| Remerciement .....                                                                         | 3  |
| Glossaire .....                                                                            | 4  |
| Table des matières .....                                                                   | 6  |
| 1. Présentation .....                                                                      | 8  |
| 1.1. L'entreprise Magellium .....                                                          | 8  |
| 1.2. L'agence de Ramonville-Saint-Agne .....                                               | 9  |
| 1.3. Positionnement stratégique .....                                                      | 9  |
| 2. Analyse du Fonctionnement et du Management de l'équipe .....                            | 10 |
| 2.1. Structure de l'équipe .....                                                           | 10 |
| 2.2. Organisation de l'équipe .....                                                        | 10 |
| 2.3. Place occupée dans le service .....                                                   | 10 |
| 2.3.1. Responsabilités et positionnement .....                                             | 10 |
| 2.3.2. Rôle et périmètre d'intervention .....                                              | 11 |
| 2.3.3. Autonomie et force de proposition technique .....                                   | 11 |
| 2.3.4. Standardisation et transmission des connaissances .....                             | 12 |
| 2.3.5. Dynamique collaborative et rituels agiles .....                                     | 12 |
| 3. Travail Réalisé : Optimisation et R&D Agentique .....                                   | 13 |
| 3.1. Axe 1 : Refonte de la stratégie de tests et optimisation CI/CD .....                  | 13 |
| 3.1.1. Contexte technique, investigation et définition des objectifs .....                 | 13 |
| 3.1.2. Les symptômes initiaux .....                                                        | 13 |
| 3.1.3. Phase d'audit et analyse des goulots d'étranglement .....                           | 13 |
| 3.1.4. Définition des objectifs .....                                                      | 14 |
| 3.1.5. Centralisation et optimisation du contexte Spring Boot .....                        | 15 |
| 3.1.6. Externalisation des contrats d'API avec Bruno .....                                 | 16 |
| 3.1.7. Fiabilisation Frontend .....                                                        | 18 |
| 3.1.8. Benchmark Technique : Cypress vs Playwright .....                                   | 18 |
| 3.1.9. Stratégie « Shift-Left » .....                                                      | 18 |
| 3.2. Axe 2 : R&D et Implémentation d'une IA Agentique Autonome .....                       | 19 |
| 3.2.1. 3.2.1. Contexte Technique, Modèles Locaux (Gemma 4) et Cas<br>d'Usage Ciblés .....  | 19 |
| 3.2.2. Infrastructure d'ingestion et interopérabilité : les fondations du<br>système ..... | 20 |
| 3.2.3. Orchestration Multi-Agents avec LangGraph .....                                     | 22 |
| 3.2.4. Architecture Orientée Objet & Robustesse des Agents .....                           | 23 |
| 3.2.5. Déploiement, Conteneurisation et Traçabilité .....                                  | 23 |
| 3.3. Axe Transverse : Industrialisation et Pattern SafeHydrator .....                      | 23 |
| 3.3.1. Conception du GenericHydrator .....                                                 | 23 |
| 3.3.2. Fiabilisation et Testabilité .....                                                  | 24 |
| 3.3.3. Chronologie et Planning de la Mission .....                                         | 24 |

|                                                                                         |    |
|-----------------------------------------------------------------------------------------|----|
| 4. Synthèse et Bilan .....                                                              | 25 |
| 4.1. Bilan technique et résultats obtenus .....                                         | 25 |
| 4.2. Estimation des gains pour l'entreprise .....                                       | 25 |
| 4.3. Bilan personnel .....                                                              | 25 |
| 5. Conclusion .....                                                                     | 26 |
| 5.1. Synthèse globale .....                                                             | 26 |
| 5.2. Perspectives .....                                                                 | 26 |
| 6. Annexes .....                                                                        | 27 |
| 6.1. Annexe 1 : Tableau d'Analyse Réflexive des Compétences (Obligatoire<br>UTBM) ..... | 27 |
| 6.2. Annexe 2 : Bibliographie & Sources Techniques (Indispensable UTBM) .               | 27 |

# 1. Présentation

## 1.1. L'entreprise Magellium

Magellium, membre fondateur du *Magellium Artal Group*, est une entreprise de taille intermédiaire (ETI) spécialisée dans le développement de solutions technologiques de pointe. Ses cœurs de métier s'articulent principalement autour de la géographie numérique, du traitement d'images et de l'ingénierie logicielle avancée. Fondée en 2003 à Toulouse, Magellium a su développer une expertise unique qui la positionne aujourd'hui comme un acteur incontournable auprès des grands donneurs d'ordres des secteurs du Spatial, de la Défense, de la Sécurité, de l'Énergie et de l'Environnement.

Le groupe compte aujourd'hui plus de 250 collaborateurs répartis sur plusieurs pôles stratégiques en France, notamment à La Garenne-Colombes (région parisienne), Nantes et Ramonville-Saint-Agne (près de Toulouse). C'est au sein de cette agence toulousaine, implantée au cœur de l'écosystème aéronautique et spatial du Sud-Ouest, que s'est déroulé ce stage de fin d'études.

Magellium collabore quotidiennement avec une clientèle diversifiée et exigeante, allant des institutions publiques majeures (comme le CNES ou le CEA) aux grands groupes industriels privés et aux organismes de recherche internationaux. Cette double culture, mêlant rigueur scientifique et agilité industrielle, permet à l'entreprise de concevoir des architectures capables de répondre à des défis technologiques hautement complexes et à forte criticité.

L'offre de Magellium s'organise et se déploie autour de quatre grands domaines d'excellence :

- **Géomatique et cartographie (SIG) :** Magellium développe des solutions avancées de cartographie numérique, de gestion de données géospatiales massives et d'analyse spatiale. L'entreprise maîtrise l'ensemble de la chaîne de valeur du domaine, de l'intégration d'outils *Open Source* (comme PostGIS ou GeoServer) au déploiement de Systèmes d'Information Géographique d'envergure.
- **Traitement d'images et Observation de la Terre :** Reconnue pour son savoir-faire applicatif, l'entreprise conçoit des chaînes complexes d'analyse de flux visuels, notamment satellitaires (comme les programmes Copernicus et Sentinel). Ces algorithmes interviennent dans la surveillance environnementale, l'étude du climat ou la reconnaissance automatique de cibles en milieu contraint.
- **Ingénierie logicielle et Cloud :** Pour soutenir ces applications lourdes, les équipes de Magellium conçoivent des architectures logicielles sur mesure, scalables et résilientes. L'accent est mis sur l'adoption des pratiques DevOps,

l'automatisation des pipelines d'intégration continue (CI/CD) et la conteneurisation des services.

- **Intelligence Artificielle et Agents Autonomes** : En réponse aux évolutions majeures du secteur, l'entreprise intègre des capacités d'intelligence artificielle, de machine learning et de deep learning. Ces briques permettent d'automatiser l'analyse prédictive, la classification d'images à la volée, et s'ouvrent activement sur l'orchestration de systèmes IA et d'agents autonomes.

## **1.2. L'agence de Ramonville-Saint-Agne**

## **1.3. Positionnement stratégique**

## 2. Analyse du Fonctionnement et du Management de l'équipe

### 2.1. Structure de l'équipe

Le département CNAPPS (Cloud, Network, Architecture, Performance, Security & Software) de Magellium est organisé en plusieurs pôles d'expertise, chacun dédié à un domaine technologique spécifique. Ces pôles sont structurés de manière à favoriser la collaboration interdisciplinaire tout en permettant une spécialisation approfondie dans les domaines clés de l'entreprise. Parmi ces pôles, on trouve notamment l'expertise en ingénierie logicielle, le pôle infrastructure et Cloud, ainsi que les unités dédiées à l'intelligence artificielle et aux systèmes autonomes.

### 2.2. Organisation de l'équipe

L'équipe au sein de laquelle j'ai évolué est structurée de manière à favoriser la collaboration et l'efficacité opérationnelle. Elle est composée de plusieurs membres aux compétences complémentaires, répartis selon les rôles suivants :

- **Chef de projet** : Responsable de la coordination globale, de la planification des tâches et de la communication avec les parties prenantes. Il joue un rôle clé dans la gestion des ressources et la prise de décision stratégique.
- **Développeurs** : Chargés de la conception, du développement et de la maintenance des solutions logicielles. Ils travaillent en étroite collaboration pour assurer la cohérence et la qualité du code produit.

Afin de coordonner les efforts et de garantir une progression fluide, des réunions quotidiennes (Daily Scrum) sont organisées. Ces échanges permettent de synchroniser l'avancement des tâches, d'identifier les obstacles et de réaligner les objectifs. Par ailleurs, des outils de gestion de projet comme Jira sont utilisés pour le suivi des tickets, l'assignation des responsabilités et la transparence de l'information.

La gestion du versionnage est assurée par Git, structuré autour de branches dédiées par fonctionnalité. Cette approche permet de maintenir un historique clair et de faciliter les revues de code, lesquelles sont systématiques pour garantir la qualité et favoriser le partage des connaissances.

### 2.3. Place occupée dans le service

#### 2.3.1. Responsabilités et positionnement

En tant qu'étudiant ingénieur en dernière année, ma mission au sein du projet Farmstar s'est articulée autour de deux axes majeurs : l'optimisation des

performances de l'infrastructure existante et la conception R&D d'un système d'intelligence artificielle autonome.

Ce double positionnement m'a conféré des responsabilités techniques importantes. Je rendais compte de mes avancées à mon tuteur technique (Mathieu NIORD), avec qui je validais les choix architecturaux critiques. Mon rôle consistait non seulement à implémenter des solutions, mais également à auditer le code existant, identifier les goulots d'étranglement (tels que les temps de build CI) et proposer des refontes structurelles. Les propositions que je développais devaient ainsi être validées pour leur faisabilité, leur sécurité et leur adéquation avec les standards de Magellium.

### 2.3.2. Rôle et périmètre d'intervention

Mon périmètre d'action s'est étendu sur l'ensemble du cycle de vie logiciel, allant du backend jusqu'à l'orchestration de modèles de langage :

- **DevOps et Qualité logicielle** : J'ai mené des audits de performance sur les pipelines d'intégration continue (CI/CD) sous GitLab, intégrant des hooks de pré-commit et pré-push (SonarLint, Checkstyle, Spotless) pour garantir la robustesse du code.
- **Ingénierie Backend et Tests** : J'ai travaillé sur la refactorisation de composants en Java (via de la réflexion) et mené la migration complète des tests d'API vers de nouveaux standards (Bruno pour l'intégration, Playwright pour l'E2E frontend).
- **R&D en IA Agentique** : J'ai conçu de bout en bout une infrastructure locale basée sur LangGraph et le Model Context Protocol (MCP). J'ai configuré des workflows multi-agents capables d'analyser du code source, de générer des plans de tests et de rédiger des documentations de manière autonome.

### 2.3.3. Autonomie et force de proposition technique

L'une des caractéristiques principales de ce stage a été la grande autonomie qui m'a été accordée dans mes recherches. Cette liberté s'est traduite par des prises de décision impactantes :

- **Benchmark et migration d'outils** : Face aux lenteurs des tests End-to-End côté frontend, j'ai initié le benchmarking de Playwright face à Cypress. Ce choix a permis de diviser le temps d'exécution par trois (de 5m28s à 1m45s) et d'améliorer la stabilité de la suite de tests.
- **Choix architecturaux en IA** : Lors du développement de l'assistant IA, j'ai identifié les limites d'isolation des agents sous Open WebUI. J'ai alors proposé et implémenté une migration vers LangGraph, introduisant un superviseur centralisé et un routage dynamique orienté objet (POO) pour permettre une communication fluide entre agents.

- **Optimisation des ressources** : Mes initiatives sur le profilage du cache Spring Boot ont permis de réduire le temps de build local de 15 minutes à seulement 4 minutes, améliorant ainsi considérablement la vélocité de l'équipe.

#### **2.3.4. Standardisation et transmission des connaissances**

Ne me limitant pas à l'écriture de code, j'ai pris le rôle de facilitateur technique. J'ai rédigé des guides de bonnes pratiques (au format Typst et Markdown) et enrichi le wiki du projet sur les standards d'implémentation des tests et l'utilisation des outils d'analyse statique.

La restitution a été une composante essentielle de ma mission. J'ai conçu et animé plusieurs présentations techniques formelles (notamment 28 slides sur la transformation de l'IA vers l'agentique et une restitution sur l'optimisation de la CI et de la rédaction de tests), permettant d'évangéliser ces nouvelles pratiques architecturales auprès de l'équipe.

#### **2.3.5. Dynamique collaborative et rituels agiles**

La réussite de ces chantiers repose sur une communication transparente :

- Des points de synchronisation réguliers pour remonter les blocages techniques et valider les Proof of Concept (PoC).
- Une démarche d'intégration continue stricte via GitLab, où chaque évolution faisait l'objet de revues de code croisées.
- Une approche itérative orientée « Fast-Fail », permettant d'ajuster rapidement l'architecture logicielle en fonction des contraintes de sécurité et des retours terrains.

## 3. Travail Réalisé : Optimisation et R&D Agentique

### 3.1. Axe 1 : Refonte de la stratégie de tests et optimisation CI/CD

#### 3.1.1. Contexte technique, investigation et définition des objectifs

À mon arrivée au sein des équipes travaillant sur le projet Farmstar, le cycle de développement logiciel était ralenti par une infrastructure de tests manquant de standardisation et d'optimisation structurelle. Bien que la base de code soit fonctionnelle et réponde aux exigences métiers, la validation de nouvelles fonctionnalités souffrait de goulots d'étranglement techniques affectant directement la productivité et la boucle de feedback des développeurs.

#### 3.1.2. Les symptômes initiaux

Lors de mes premières semaines de prise en main du projet, trois problématiques majeures se sont dégagées :

- **Lenteur prononcée de l'Intégration Continue (CI) :** Le temps d'exécution des tests du module « fs-core » s'avérait particulièrement long, s'étalant de 17 à 20 minutes en local, et atteignant jusqu'à 40 minutes sur les pipelines GitLab CI.
- **Complexité et effet « boîte noire » :** Les tests d'intégration backend étaient utilisés de manière indifférenciée, à la fois pour valider la logique métier complexe et pour vérifier de simples contrats d'API (tests End-to-End). Cette confusion des périmètres rendait la suite de tests verbeuse, et il était difficile de distinguer les tests réellement critiques des tests de « surface ».
- **Fragilité de la suite de tests du backend :** Il n'existait pas de stratégie de tests du fumée (**smoke tests**) pour les routes backend empruntés par le frontend. Ainsi, une modification du backend pouvait passer inaperçue si elle n'était pas couverte par un test d'intégration, exposant le projet à des risques de régressions non détectées lors des mises en production.
- **Fragilité de l'environnement frontend :** Les tests de fumée (**smoke tests**) permettant de valider l'état de des routes utilisés côté frontend après déploiement étaient inexistantes, exposant le projet à des risques de régressions non détectées lors des mises en production.

#### 3.1.3. Phase d'audit et analyse des goulots d'étranglement

Pour comprendre l'origine exacte de ces lenteurs, notamment sur la partie backend, une phase d'investigation technique a été initiée. Il était nécessaire de dépasser le simple constat de lenteur pour diagnostiquer le comportement de la JVM durant l'exécution des suites de tests.

J'ai procédé au déploiement d'outils d'audit de performance de l'application, et plus particulièrement de l'outil de métrologie « spring-test-profiler ». L'analyse

attentive des rapports générés a mis en lumière une anomalie architecturale : un manquement dans la gestion du cache Spring.

Les métriques ont révélé que chaque classe de test provoquait la destruction et le redémarrage complet du contexte applicatif. Au lieu de réutiliser un environnement partagé, la JVM était forcée de recharger un monolithe de plus de 2000 beans à chaque itération. Cette « fuite de contexte » imposait une pénalité systématique d'environ 45 secondes par démarrage. Il est alors apparu évident que la source principale de l'effondrement des performances de la CI n'était pas liée à la complexité des tests eux-mêmes, mais bien à la charge d'initialisation répétée du framework.

#### 3.1.4. Définition des objectifs

Suite à ce diagnostic précis, les objectifs de cette première phase de mission ont été définis afin de restructurer la stratégie de test en profondeur. L'enjeu était de résoudre les anomalies identifiées sans altérer la couverture fonctionnelle du code. Les axes d'amélioration ont été fixés ainsi :

##### 1. Résolution des anomalies de performance

- Identifier et éliminer les configurations provoquant la fuite de contexte Spring mise en évidence lors de l'audit.
- Centraliser le chargement de l'environnement applicatif pour garantir un **Cache Hit Rate** maximal et réduire le temps d'exécution global à quelques minutes.

##### 2. Clarification de l'architecture de test

- Séparer strictement les responsabilités : isoler la validation de la logique métier interne de la validation des points de terminaison HTTP.
- Déporter la vérification des contrats d'API vers un outillage externe dédié, léger et exempt du coût de démarrage du backend. (Tests d'intégration vs Tests End-to-End vs Smoke Tests)
- Combler le déficit de couverture frontend en déployant une infrastructure de tests de non-régression rapide et isolée des problématiques réseau. (Smoke Tests Frontend)

##### 3. Sécurisation « Shift-Left » et Standardisation

- Déplacer la vérification de la qualité et de la sécurité du code au plus près du développeur, en interceptant les anomalies avant même le déclenchement de la CI. (Pré-commit et pré-push Git Hooks)
- Rédiger un référentiel technique documentant ces nouvelles normes afin de standardiser l'écriture des tests au sein de l'équipe et de prévenir l'accumulation future de dette technique. (Guide de bonnes pratiques, Wiki interne)

Pour répondre aux objectifs fixés, la refonte s’est articulée autour de trois chantiers techniques majeurs, allant du cœur du backend jusqu’à la chaîne d’intégration continue.

### 3.1.5. Centralisation et optimisation du contexte Spring Boot

Le premier défi consistait à endiguer la « fuite de contexte » identifiée lors de l’audit. J’ai conçu et mis en place une classe mère globale, « `AbstractIntegrationTest` », dont héritent désormais toutes les classes de test. Cette architecture a permis de mutualiser les configurations complexes (« `MockBean` », « `SpyBean` ») et d’éliminer les annotations destructrices de cache, telles que « `DirtyContext` ».

Parallèlement, la gestion de la sécurité asynchrone levait fréquemment des exceptions (« `AccessDeniedException` »). J’ai développé un utilitaire centralisé (« `SecurityTestUtils` ») pour forcer l’injection de contextes de sécurité fiables et liés à la base de données de test, remplaçant ainsi les annotations magiques devenues instables.

**Résultat :** Le temps de build local est passé de 17 minutes à 3 minutes, avec un taux d’utilisation du cache (**Cache Hit Rate**) stabilisé à 99,5%. Sur la CI GitLab, le temps d’exécution a été réduit de 70%, passant de 19:18 à 5:45.

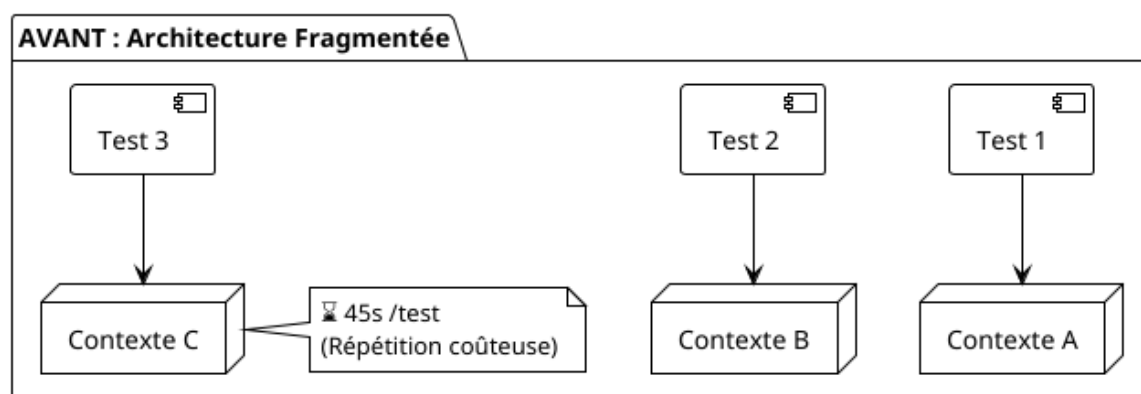


Fig. 1. – Avant Optimisation : Chaque classe de test réinitialise le contexte Spring Boot, entraînant des pénalités de performance.

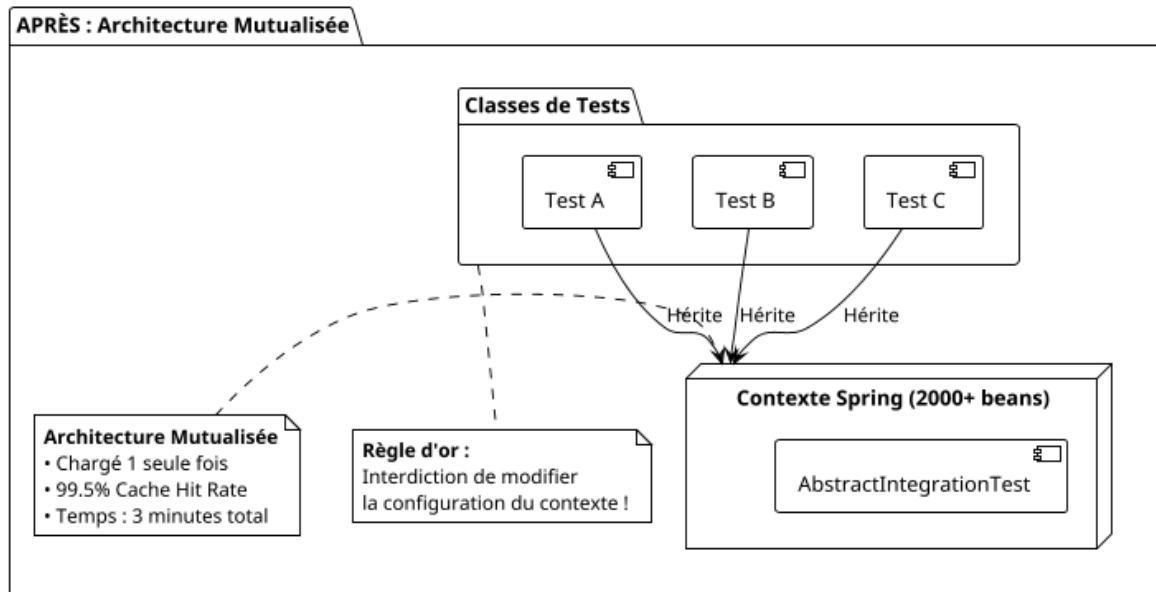


Fig. 2. – Après Optimisation : Mutualisation du contexte Spring Boot pour tous les tests d'intégration.

### 3.1.6. Externalisation des contrats d'API avec Bruno

Pour désengorger les tests d'intégration Spring, j'ai migré des tests de validation HTTP (« MockMvc ») vers le client d'API Bruno. Afin de rendre ces tests de bout en bout autonomes et rejouables sans polluer la base de données de la CI, j'ai une stratégie de génération d'IDs dynamiques via des scripts JavaScript pré-requêtes (génération de timestamps et UUID). Enfin, j'ai ajouté au script Bash d'exécution des tests bruno sur la CI, le run de cette nouvelle suite de tests.

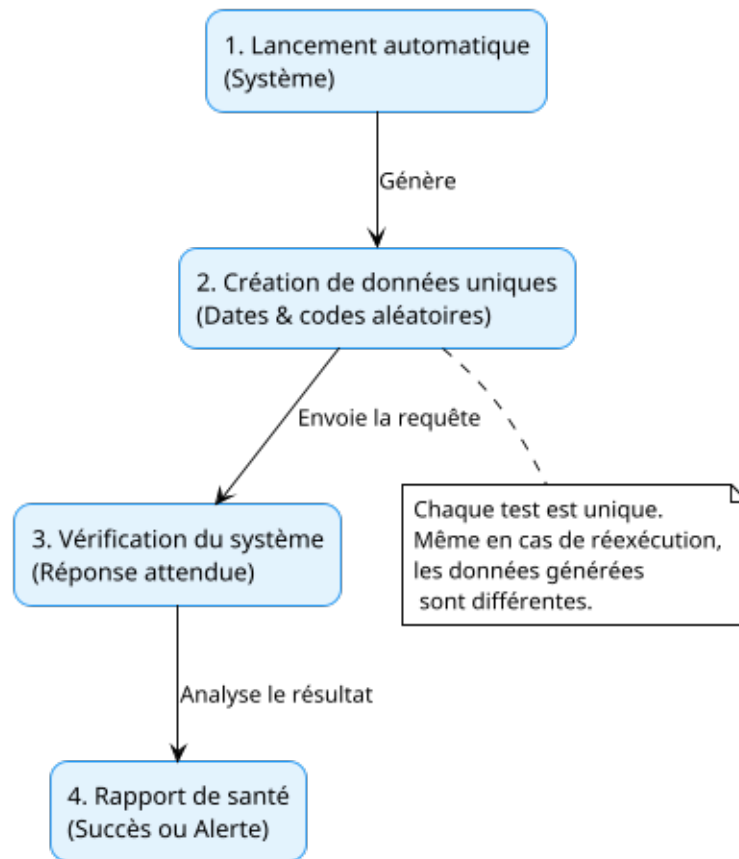


Fig. 3. – Workflow de validation des contrats d’API via Bruno

J’ai créé une suite de smoke tests couvrant l’intégralité des routes API backend sollicitées par le frontend lors de son chargement.

**Objectif :** Vérifier que chaque point d’entrée métier (Identité, Agronomie, Gestion de parcelles) répond aux attentes contractuelles du frontend. Ces tests agissent comme une sentinelle : ils permettent de détecter immédiatement si une modification backend « casse » la communication avec l’interface.

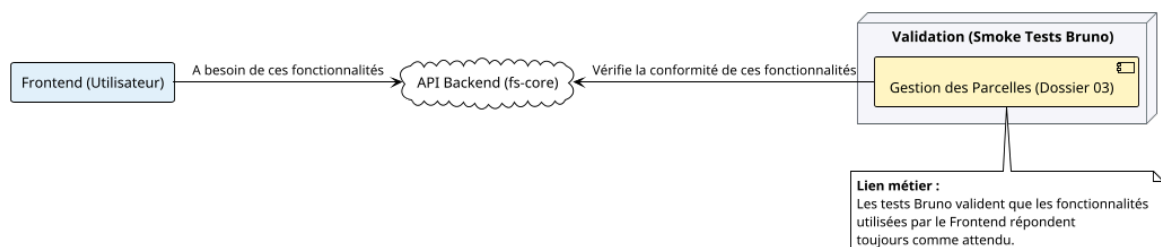


Fig. 4. – Quand est-ce qu’il s’agit d’un smoke test ?

### 3.1.7. Fiabilisation Frontend

Côté frontend, j'ai mis en place une suite de tests de fumée (**Smoke Tests**) pour valider les routes critiques utilisées par l'interface utilisateur. Afin de choisir quel framework de tests automatisés utiliser, j'ai mené un benchmark technique comparant Cypress et Playwright. Les résultats ont montré que l'infrastructure existante sous Cypress était trop lourde pour les besoins du projet. (TODO : insérer benchmark technique Cypress vs Playwright) Une fois le choix de Playwright validé, j'ai réécrit les tests de fumée pour qu'ils soient plus rapides et plus fiables. En cours de développement, il s'est avéré que la dépendance de ces tests au lancement du backend ralentissait considérablement le cycle de la CI. Ainsi, j'ai mis en place un mock du backend pour isoler les tests frontend et réduire le temps d'exécution. Cette approche a permis de diviser le temps d'exécution des Smoke Tests par trois (de 5m 28s à 1m 45s).

### 3.1.8. Benchmark Technique : Cypress vs Playwright

Pour valider l'architecture de tests de fumée frontend du frontend, une étude comparative a été menée sur une suite représentative de 18 parcours utilisateurs critiques (Identity, Agronomy, Parcel Management, Monitoring).

| Métrique / Critère           | Cypress<br>(Existant) | Playwright<br>(Cible)    | Gain / Impact       |
|------------------------------|-----------------------|--------------------------|---------------------|
| Temps d'exécution global     | 5 min 28 s            | 1 min 45 s               | Temps divisé par 3  |
| Consommation CPU (User time) | 1 min 38 s            | 8.5 s                    | Réduction de 91%    |
| Taux de succès de la suite   | 88,8 % (flaky tests)  | 100 %                    | Stabilité maximale  |
| Architecture d'exécution     | Proxy réseau lourd    | Protocole DevTools natif | Faible empreinte CI |

L'avantage de Playwright réside dans sa capacité à intercepter nativement le réseau via une fonction dédiée (« injectAuthHeaders »), injectant de manière dynamique les en-têtes d'authentification (« x-authenticated-user », « x-authenticated-roles ») sans dépendre d'un sous-système complexe. De plus, le découplage des tests frontend vis-à-vis du backend via un système de moquage par fichiers « .har » a permis de sécuriser le build sur la CI GitLab.

### 3.1.9. Stratégie « Shift-Left »

Pour garantir la pérennité de ces optimisations, j'ai mis en place une stratégie « Shift-Left » via des hooks Git. J'ai configuré des scripts « pre-commit » (exécutant Spotless, Checkstyle et SonarLint uniquement sur les fichiers modifiés) et « pre-

push » (lançant les Smoke Tests impactés). Cela permet d’intercepter les régressions directement sur le poste du développeur, avant de solliciter la CI.

### 3.2. Axe 2 : R&D et Implémentation d’une IA Agentique Autonome

#### 3.2.1. 3.2.1. Contexte Technique, Modèles Locaux (Gemma 4) et Cas d’Usage Ciblés

L’arrivée de modèles de langage open-source à capacités avancées (tels que Gemma 4, décliné de 2.3B à 31B de paramètres) marque un tournant pour les déploiements **on-premise** en entreprise. Offrant une fenêtre de contexte native allant jusqu’à 128k tokens, ces modèles permettent d’analyser des blocs de code complexes tout en garantissant la confidentialité absolue des données de Magellium.

Toutefois, le codebase du projet Farmstar dépassant largement cette limite de mémoire immédiate, une simple injection brute entraînerait une saturation de la VRAM GPU et un effondrement des performances. Une démarche de **Context Engineering** a donc été formalisée autour de 5 cas d’usage prioritaires :

- 1. **Assistant de Code en temps réel** : Binôme de programmation suggérant des implémentations respectant les normes **Clean Code** du projet.
- 2. **Workflow Ticket Jira** : Analyse automatique des tickets, identification des modules impactés (Front/Back) et génération de plans d’action.
- 3. **Revue de Code automatisée** : Premier filtre sur les Merge Requests GitLab pour vérifier la conformité aux standards avant la revue humaine.
- 4. **Génération de Tests** : Création automatique de suites unitaires et d’intégration (JUnit 5 / Mockito) couvrant les cas aux limites (**edge cases**).
- 5. **Rétro-Ingénierie** : Exploration interactive du code legacy pour reconstituer l’architecture et la logique métier sans documentation préalable.

| Modèle Cible  | VRAM Minimale | Profil d’Usage & Inférence Local                                              |
|---------------|---------------|-------------------------------------------------------------------------------|
| Gemma 2B / 7B | 4 à 12 GB     | Assistant de code réactif sur poste développeur (RTX 4070).                   |
| Gemma 13B     | 16 à 20 GB    | Analyse de modules et génération de tests unitaires (RTX 3090).               |
| Gemma 31B     | 24+ GB        | Orchestration complexe, revue de PR et raisonnement multi-agents (Multi-GPU). |

### **3.2.2. Infrastructure d'ingestion et interopérabilité : les fondations du système**

Pour que l'architecture multi-agents puisse interagir efficacement avec la base de code de Farmstar, il était impératif de construire une infrastructure de base robuste, capable d'extraire le contexte technique, de fournir des outils exploitables et d'être déployée de manière reproductible.

### **Conception d'un module d'ingestion générique :**

Afin que l'IA puisse « lire » et comprendre le projet, j'ai conçu et développé un module d'ingestion générique. Dans la version développée pour le PoC, ce module prend en charge l'extraction et le traitement de fichiers aux formats « .java », « .py », « .txt » et autres, tout en ayant été pensé pour être facilement extensible à de nouveaux formats à l'avenir. Pour garantir la fiabilité de ce processus d'ingestion, un système de **logging** a été mis en place afin de tracer précisément quels fichiers ont été traités et de faciliter le débogage.

De plus, une harmonisation stricte des chemins d'accès a été réalisée entre le module d'ingestion et le serveur MCP. Cette standardisation s'est avérée cruciale : elle a considérablement amélioré la lisibilité du contexte pour les agents LangGraph et a optimisé l'utilisation des outils de recherche automatisés (tels que « regex » et « read\_file »).

### **Conteneurisation et orchestration (Docker) :**

Afin de garantir l'isolation des environnements et de prévenir les conflits de dépendances Python inhérents à la juxtaposition de plusieurs briques technologiques, l'ensemble du PoC a été dockerisé. Des « Dockerfiles » spécifiques ont été rédigés pour chaque composant clé : l'API d'ingestion, le serveur MCP, le moteur LangGraph et l'interface utilisateur. L'orchestration de ces services est assurée par un fichier « docker-compose.yml », simplifiant drastiquement le processus de démarrage et validant l'interopérabilité des conteneurs dans un environnement isolé.

**Interface d'administration (Streamlit)** Pour piloter cette infrastructure logicielle de manière ergonomique, j'ai développé une application web d'administration en utilisant le framework Streamlit. Cette interface permet de configurer dynamiquement le système, offrant la possibilité de définir le chemin absolu du répertoire à ingérer et de nommer la collection de données cible sans avoir à modifier les fichiers de configuration manuellement.

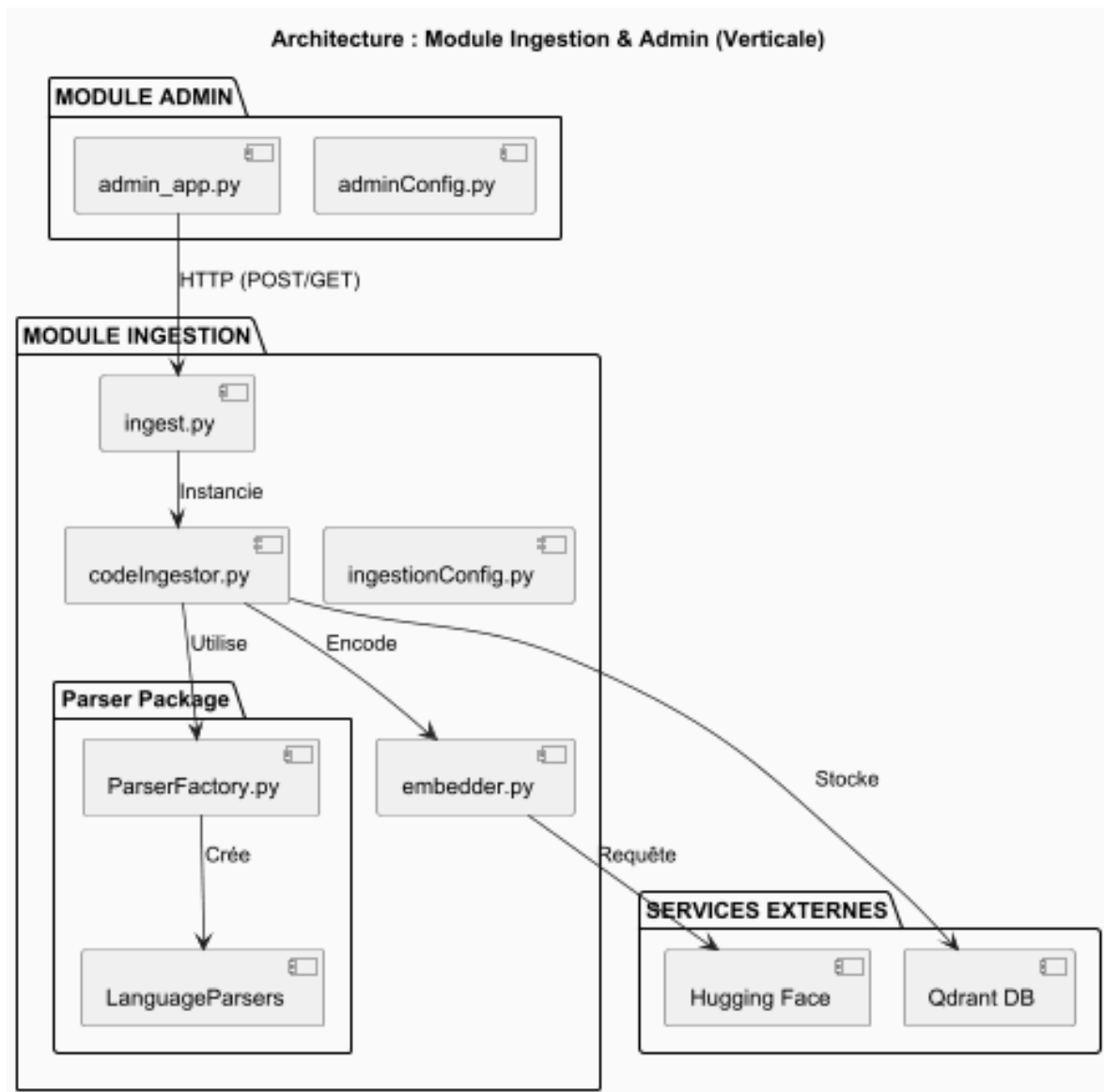


Fig. 5. – Architecture du système : Module d’ingestion et interface d’administration Streamlit

### 3.2.3. Orchestration Multi-Agents avec LangGraph

Pour surmonter les limitations d’isolation des chatbots traditionnels (Open WebUI), l’architecture a évolué vers un graphe d’agents orchestré par **LangGraph**. Le système repose sur un agent **Superviseur** agissant comme point d’entrée unique et cerveau décisionnel.

#### Architecture du Graphe Principal :

- **Superviseur Central** : Analyse l’intention utilisateur via une sortie structurée Pydantic et route la requête vers le sous-graphe adéquat.
- **Détection de contexte autonome** : Détecte si du code ou des rapports de couverture (**gaps**) sont fournis dans le prompt pour adopter un routage court-circuit.

- **Sous-graphe TEST\_COVERAGE** : Organisé en trois rôles séquentiels mais modulaires :
  - **Gatherer** : Identifie et extrait le code source et les tests existants via le serveur MCP.
  - **Analyzer** : Évalue les écarts de couverture et établit un plan de test.
  - **Solver** : Génère les suites de tests unitaires (JUnit 5 / Mockito) correspondantes.

#### 3.2.4. Architecture Orientée Objet & Robustesse des Agents

Afin de garantir la maintenabilité et la qualité logicielle du PoC, le framework d'agents a été refactorisé selon les principes de la programmation orientée objet (POO) :

- **Classe Abstraite « Agent » (ABC)** : Encapsule le cycle de vie de chaque nœud, le chargement dynamique des configurations (« prompts/ » et « skills/ »), et l'accès au registre d'outils MCP.
- **Typage fort et Registres** : Utilisation d'énumérations Python (« SubGraphType », « SubAgentType », AgentTool) pour éliminer les chaînes de caractères brutes dans le routage.
- **Génération de Schémas Dynamiques** : Utilisation des bibliothèques **Pydantic** et **Instructor** (« pydantic.create\_model ») pour forcer le LLM à retourner des formats JSON stricts et typés, intégrant des mécanismes de retry automatique en cas de non-conformité.

#### 3.2.5. Déploiement, Conteneurisation et Traçabilité

Afin de rendre ce prototype facilement déployable et exploitable par les développeurs de l'équipe Farmstar, l'ensemble de la suite a été conteneurisé sous Docker :

- **Docker Compose** : Orchestre les microservices (API d'ingestion, serveur FastMCP, moteur LangGraph et UI Streamlit) dans un réseau isolé.
- **Observabilité** : Intégration de la plateforme **LangFuse** pour mesurer la latence, tracer les appels d'outils inter-agents, et suivre l'utilisation des tokens.

### 3.3. Axe Transverse : Industrialisation et Pattern SafeHydrator

En parallèle des chantiers de CI et d'IA, un travail d'architecture logicielle a été mené sur le backend Java (fs-core) pour simplifier les services métier complexes (tels que StrategyService).

#### 3.3.1. Conception du GenericHydrator

La création et la mise à jour d'entités JPA complexes nécessitaient une vérification répétitive des DTOs. J'ai développé un composant générique GenericHydrator s'appuyant sur la réflexion Java :

- `@HydrationField` : Définie sur les DTOs pour automatiser le mapping vers les entités.
- `@HydrationGuard` : Placé sur les méthodes de validation métier (ex: vérification des doublons de noms, règles d'intervalles de dates).

### 3.3.2. Fiabilisation et Testabilité

Pour valider ce composant sans dupliquer de code de test, j'ai mis en place une classe abstraite de test (`AbstractGenericHydratorTest`) exploitant la puissance des tests paramétrés JUnit 5 (`@ParameterizedTest`, `@MethodSource`). Cette approche a permis de couvrir l'intégralité des cas aux limites (**edge cases**) et d'assurer une remontée propre des exceptions métier (converties en erreurs HTTP 400).

### 3.3.3. Chronologie et Planning de la Mission

| Période             | Planning Prévisionnel                    | Réalisation Réelle                                                        |
|---------------------|------------------------------------------|---------------------------------------------------------------------------|
| Février 2026        | Prise en main fs-core & audit des builds | Refactoring<br><code>AbstractIntegrationTest</code><br>& profilage Spring |
| Mars 2026           | Migration des tests API & CI             | Portage Bruno, Playwright<br>Frontend & Hooks Git                         |
| Avril 2026          | R&D IA & FastMCP                         | Setup Open WebUI,<br>FastMCP & Pattern<br><code>SafeHydrator</code>       |
| Mai 2026            | Orchestration LangGraph                  | Migration Multi-agents<br>POO & Subgraphs                                 |
| Juin - Juillet 2026 | Industrialisation PoC & Rédaction        | Module Ingestion<br>générique, Dockerisation &<br>Rapport ST50            |

## 4. Synthèse et Bilan

### 4.1. Bilan technique et résultats obtenus

L'ensemble des objectifs fixés au début du stage a été atteint, apportant des améliorations mesurables sur le projet Farmstar :

- **Intégration Continue (CI/CD) :** Temps d'exécution du build backend réduit de 70% sur GitLab CI (de 19:18 à 5:45 min), et build local accéléré de 17 min à 3 min grâce à un taux d'utilisation du cache Spring de 99,5%.
- **Tests Frontend :** Suite de smoke tests Playwright 3 fois plus rapide que Cypress (1 min 45 s vs 5 min 28 s).
- **Qualité du code :** Interception des régressions avant la CI via des hooks Git pre-commit et pre-push ciblés sur les fichiers modifiés.
- **R&D IA Agentique :** Réalisation d'un PoC multi-agents opérationnel basé sur LangGraph et MCP, capable d'analyser le code source et de générer des plans de tests.

### 4.2. Estimation des gains pour l'entreprise

- **Productivité Développeur :** L'accélération des builds et la réduction de la boucle de feedback font gagner en moyenne 30 à 45 minutes par jour et par développeur sur l'équipe Farmstar.
- **Réduction des coûts d'infrastructure :** La baisse de la durée des jobs GitLab CI réduit directement le temps de calcul des runners.
- **Pérennité du Code :** La standardisation des pratiques de tests et les guides rédigés sur le wiki garantissent une baisse durable de la dette technique.

### 4.3. Bilan personnel

Ce stage de fin d'études a été une expérience d'ingénieur complète. Sur le plan technique, il m'a permis d'approfondir mes connaissances des entrailles de Spring Boot, du scripting CI/CD avancé, et des architectures émergentes d'IA agentique (LangGraph, MCP). Sur le plan méthodologique, j'ai développé ma capacité à mener des audits de performance, à prendre des décisions d'architecture en autonomie, et à communiquer mes choix à travers des présentations techniques et de la documentation.

## 5. Conclusion

### 5.1. Synthèse globale

Ce projet de fin d'études au sein de l'agence Magellium s'inscrit dans une démarche globale d'optimisation de l'ingénierie logicielle. Qu'il s'agisse de résoudre les problématiques de performance de la CI/CD ou de concevoir l'architecture IA de demain, l'approche retenue a toujours privilégié la rigueur architecturale, la qualité de code et l'autonomie des développeurs.

### 5.2. Perspectives

Plusieurs axes d'évolution peuvent prolonger ces travaux :

1. **Industrialisation du PoC IA** : Connecter l'orchestrateur LangGraph directement aux pipelines GitLab de Magellium pour automatiser les revues de code et l'audit de sécurité sur chaque Merge Request
2. **Extension des Serveurs MCP** : Ajouter des intégrations directes vers les APIs Jira et SonarQube pour permettre à l'agent de résoudre des tickets de bout en bout.
3. **Généralisation du pattern Hydrateur** : Étendre le GenericHydrator aux autres microservices du projet Farmstar.

## 6. Annexes

### 6.1. Annexe 1 : Tableau d'Analyse Réflexive des Compétences (Obligatoire UTBM)

| Domaine de Compétence                | Compétences Mobilisées / Développées                                                                                       | Projets / Actions Concrètes                                                                             |
|--------------------------------------|----------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Ingénierie Logicielle & Architecture | Conception orientée objet, design patterns (Factory, Hydrator, Observer), refactoring de code legacy Java/Spring Boot.     | Développement du GenericHydrator, unification des contextes de test Spring (AbstractIntegrationTest).   |
| DevOps & Qualité Logicielle          | Optimisation de chaînes CI/CD, métrologie/profiling, écriture de hooks Git (pre-commit/pre-push), conteneurisation Docker. | Réduction du temps de build GitLab CI de 70%, benchmark Playwright vs Cypress, dockerisation du PoC IA. |
| Intelligence Artificielle & R&D      | Orchestration d'agents LLM, protocole MCP, RAG (Vector DB Qdrant), typage strict des sorties (Instructor/Pydantic).        | Conception de l'architecture multi-agents LangGraph et du serveur FastMCP pour le projet Farmstar.      |
| Gestion de Projet & Posture          | Autonomie, démarche « Fast-Fail », animation de présentations techniques, rédaction de documentation et wiki.              | Animation de la restitution technique (28 slides) et rédaction des guides de bonnes pratiques de test.  |

### 6.2. Annexe 2 : Bibliographie & Sources Techniques (Indispensable UTBM)

- **Spring Framework Documentation** – Spring Test Context Management & Caching, 2026.
- **LangChain & LangGraph Documentation** – Multi-Agent Orchestration & State Management, 2026.
- **Model Context Protocol (MCP) Specification** – Anthropic Open Standard for AI Tool Integration, 2025-2026.
- **Playwright Frontend Testing** – Network Interception and Mocking Strategies, Fast End-to-End Execution, 2026.
- **Instructor Python Library** – Structured Outputs for LLMs via Pydantic Validation, 2026.